



LeetCode 58: Length of Last Word

1. Problem Title & Link

Title: LeetCode 58: Length of Last Word

Link: <https://leetcode.com/problems/length-of-last-word/>

2. Problem Statement (Short Summary)

Given a string `s` consisting of words and spaces, return the length of the **last word**.

A word is defined as:

A sequence of non-space characters.

Ignore any leading or trailing spaces.

Example

Input:

`s = "Hello World"`

Output:

5

Because:

Last word = "World"

Length = 5

3. Examples (Input → Output)

Example 1

Input:

`s = "Hello World"`

Output:

5

Example 2

Input:

`s = " fly me to the moon "`

Output:

4

Last word:

moon

Example 3

Input:



s = "luffy is still joyboy"

Output:

6

Last word:

joyboy

Example 4

Input:

s = "a"

Output:

1

4. Constraints

$1 \leq s.length \leq 10^4$

s contains English letters and spaces

There is at least one word

Important:

Trailing spaces may exist.

5. Core Concept (Pattern / Topic)

String Traversal

Secondary Topics:

- Reverse Traversal
- Two Pointers

6. Thought Process (Step-by-Step Explanation)

Brute Force Idea

Split the string.

Example:

"Hello World"

Split:

["Hello", "World"]

Last word:

"World"

Length:



5

Problem

Although easy:

`split()`

creates extra space.

Interviewers often expect a direct traversal solution.

Better Observation

We only need:

Length of the last word

Not all words.

Example

Input:

"Hello World "

Notice:

Trailing spaces

must be ignored first.

Step 1

Move from right to left.

Skip spaces.

Hello World____

↑

Skip all spaces.

Step 2

Start counting characters.

Hello World

↑

Count:

d

l

r

o

W



Length:

5

Key Insight

Traverse from the end.

1. Ignore trailing spaces.
2. Count characters until space appears.

Done.

7. Visual / Intuition Diagram (ASCII Diagram)

Example:

"fly me to the moon "

Start:

fly me to the moon__

↑

Skip spaces.

fly me to the moon

↑

Count:

n

o

o

m

Length:

4

8. Pseudocode (Language Independent)

```
i = last index

while i >= 0 and s[i] == ' '
    i--

count = 0

while i >= 0 and s[i] != ' '
    count++
    i--

return count
```

9. Code Implementation



Python

```
class Solution:
    def lengthOfLastWord(self, s: str) -> int:

        i = len(s) - 1

        while i >= 0 and s[i] == ' ':
            i -= 1

        length = 0

        while i >= 0 and s[i] != ' ':
            length += 1
            i -= 1

        return length
```

Java

```
class Solution {

    public int lengthOfLastWord(String s) {

        int i = s.length() - 1;

        while(i >= 0 && s.charAt(i) == ' ')
            i--;

        int length = 0;

        while(i >= 0 && s.charAt(i) != ' ') {

            length++;
            i--;
        }

        return length;
    }
}
```

10. Time & Space Complexity

Time Complexity

$O(n)$

At most one traversal.

Space Complexity



$O(1)$

No extra storage.

11. Common Mistakes / Edge Cases

Mistake 1

Not handling trailing spaces.

Input:

"Hello World "

Answer:

5

Not:

0

Mistake 2

Using:

`s.split(" ")`

This may create empty strings.

Example:

"Hello World"

Produces:

`["Hello", "", "", "World"]`

Mistake 3

Traversing from beginning unnecessarily.

Edge Cases

Single Word

Input:

"Python"

Output:

6

Single Character

Input:

"a"

Output:

1



Many Spaces

Input:

" a "

Output:

1

12. Detailed Dry Run

Input:

s = "Hello World"

Index	Character	Action	Count
10	d	Count	1
9	l	Count	2
8	r	Count	3
7	o	Count	4
6	W	Count	5
5	Space	Stop	5

Return:

5

13. Common Use Cases (Real-Life / Interview)

Text Editors

Count length of current word.

Search Engines

Token processing.

NLP (Natural Language Processing)

Word extraction.

Log Parsing

Extract final token.

14. Common Traps (Important!)

Trap 1

Forgetting trailing spaces.

**Trap 2**

Assuming only one space between words.

Example:

"fly me to moon"

Still valid.

Trap 3

Using unnecessary extra memory.

15. Builds To (Related LeetCode Problems)

LC 58 - Length of Last Word



LC 151 - Reverse Words in a String



LC 557 - Reverse Words in a String III



LC 344 - Reverse String



LC 541 - Reverse String II

These strengthen string traversal skills.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Split	$O(n)$	$O(n)$	Easy
Reverse Traversal	$O(n)$	$O(1)$	Best

Approach 1: Split

`len(s.split()[-1])`

Very short.

Uses extra space.

Approach 2: Reverse Traversal (Chosen)

No extra memory.

Interview-preferred.



17. Why This Solution Works (Short Intuition)

The last word is the first sequence of non-space characters encountered when scanning from the end. By skipping trailing spaces and counting until a space appears, we directly obtain its length.

18. Variations / Follow-Up Questions

Follow-Up 1

Return the last word itself.

Example:

"Hello World"

Output:

"World"

Follow-Up 2

Return all words in reverse order.

Related:

LC 151

Reverse Words in a String

Follow-Up 3

Count the number of words.

Follow-Up 4

Find the longest word in a sentence.

Interview Takeaway

The key pattern is:

Reverse String Traversal

Whenever you hear:

Last Word

Last Token

Last Element from End

consider:

Start from the end and move backwards

Often this produces a cleaner and more memory-efficient solution than splitting the entire string.